East West University

Department of Computer Science and Engineering

Lab Assignment - 1

Deep Learning Model Comparison: CNN vs. Transformer Architectures

Course	: CSE 475 — Machine Learning	Instructor	: Dr. Md. Rifat Ahmmad Rashid
Type	: Group (max 4 members)	Total Marks	: 100
Deadline	: **31 March 2026, 11:59 PM**	Viva Date	: **31 March 2026**
Submission	: Kaggle Public Notebook URL (Save & Commit)		

1. Assignment Overview

This assignment requires you to implement, train, and critically evaluate state-of-the-art deep learning architectures on an image classification dataset. You will produce **two well-structured Kaggle notebooks**:

- **Notebook 1** — CNN-based models (MobileNetV3, ResNeXt-50, and EfficientNet-B3) must **be run for a minimum of 30 epochs.**
- **Notebook 2** — Transformer-based models (ViT, Swin Transformer, and DeiT) must **be run for a minimum of 30 epochs.**

  You can use the example notebook

  - **Notebook 1** — `https://www.kaggle.com/code/mrifatrashid/cse-475-cnn-architectures-deep-dive`
  - **Notebook 2** — `https://www.kaggle.com/code/mrifatrashid/cse-475-vision-transformer-exploration`

The dataset for your group has already been assigned. Each notebook must be self-contained, reproducible, and follow the exact section structure specified in this document.

> **[!] Reproducibility Requirement:** Set a global random seed at the top of each notebook. All results must be reproducible by running all cells in order with `Run All`.

2. Learning Objectives

Upon completion of this assignment, students will be able to:

1. Implement and fine-tune convolutional and transformer-based architectures using PyTorch.
2. Design robust data preprocessing and augmentation pipelines for image data.
3. Apply best practices for training infrastructure including learning rate scheduling, early stopping, and mixed-precision training.
4. Systematically compare model performance using standard evaluation metrics.
5. Conduct robustness and error analyses to derive actionable insights.
6. Communicate experimental findings in a structured, reproducible format.

3. Notebook Specifications

3.1. General Requirements

Both notebooks must adhere to the following standards:

- Written in **Python 3.10+** using PyTorch.
- Must run completely on Kaggle (GPU accelerator enabled).
- Each markdown cell should clearly label its section with the required heading.
- Code cells must be well-commented.
- All plots must have titles, axis labels, and legends where applicable.
- Model must **run for minimum 30 epochs.**
- Final notebook must be **saved, committed, and made public** on Kaggle before the deadline.

3.2. Dataset Upload Requirement

Each group must upload their assigned dataset directly to Kaggle as a **Kaggle Dataset** and use that uploaded dataset inside their notebooks. Referencing local files or external download scripts is **not permitted**.

1. Go to **kaggle.com** → **Datasets** → **New Dataset** and upload your assigned dataset.
2. Name the dataset clearly, e.g. `cse475-groupXX-dataset`.
3. The Kaggle dataset **does not need to be made public** — keeping it private (accessible only to your account) is perfectly acceptable.
4. Inside your notebook, add the dataset via **Add Data** → **Your Datasets** so it appears under `/kaggle/input/`.
5. All data loading paths in the notebook must reference `/kaggle/input/<your-dataset-name>/`.

Dataset Source Acknowledgement (Mandatory): You must clearly state the **original dataset name** and provide its **official website/source link** in two places: (1) in the Group Details cell at the top of each notebook, and (2) as a dedicated acknowledgement markdown cell placed immediately before the EDA section. Failure to cite the data source will result in a mark deduction.

3.3. Notebook Naming Convention

Each notebook **must be named exactly** according to the following format before committing:

```
CSE 475 - Assignment 01 - Group <ID>
```

Examples: `CSE 475 - Assignment 01 - Group 01` | `CSE 475 - Assignment 01 - Group 12`

Notebooks with incorrect names will not be accepted. Apply this name to **both** notebooks (CNN and Transformer).

3.4. Group Details Cell (Mandatory First Cell)

The **very first markdown cell** of each notebook must contain a group information block in the following format. This cell must appear *before* the Global Configuration section.

Required First Cell — Markdown Template

```markdown
# CSE 475 - Assignment 01
## Group Information

| Field                    | Details                              |
|-------------------------|-------------------------------------|
| **Group ID**             | Group XX                             |
| **Student 1 Name**       | Full Name                            |
| **Student 1 ID**         | 20XX-X-XX-XXX                        |
| **Student 2 Name**       | Full Name  (if applicable)           |
| **Student 2 ID**         | 20XX-X-XX-XXX (if applicable)        |
| **Notebook Type**        | CNN Models / Transformer Models      |
| **Dataset Source Name** | e.g. Oxford-IIIT Pet Dataset         |
| **Dataset Source Link** | e.g. https://www.robots.ox.ac.uk/~vgg/data/pets/ |
| **Kaggle Dataset Path** | /kaggle/input/<dataset-name>/        |
| **Submission Date**      | DD Month 2026                        |
```

3.5. Notebook 1 — CNN Models

[NB] Notebook 1: CNN-Based Architectures — MobileNetV3 · ResNeXt-50 · EfficientNet-B3

Required Section Structure (in order):

1. **[*] Global Configuration** `(START HERE)`
   Define all hyperparameters, paths, seeds, and toggles in a single cell. No hardcoded values should appear elsewhere in the notebook.

2. **[+] Setup & Imports**
   Install any required packages, import all libraries, and confirm GPU availability. Print framework versions.

3. **[=] Exploratory Data Analysis (EDA)**
   Visualize class distribution, sample images per class, image size statistics, and any relevant dataset characteristics. Identify potential imbalances or quality issues.

4. **[W] Data Preprocessing & Augmentation**
   Define training and validation/test transforms. Justify your augmentation choices with reference to the dataset characteristics observed in EDA. Include at least: resize/crop, normalization, random horizontal flip, and one advanced augmentation (e.g., CutMix, MixUp, RandAugment).

5. **[T] Training Infrastructure**
   Implement a generic training loop supporting: loss function selection, optimizer configuration (Adam / SGD + momentum), learning rate scheduling (e.g., CosineAnnealingLR, ReduceLROnPlateau), early stopping, and checkpoint saving for the best model.

6. **[O] CNN Models: MobileNetV3 · ResNeXt-50 · EfficientNet-B3**
   For **each** of the three architectures:
   - Load pretrained weights (ImageNet) and adapt the classification head.
   - State the number of trainable parameters.
   - Train using the infrastructure from Section 5.
   - Plot training and validation loss/accuracy curves.
   - Save the best checkpoint.

7. **[ ] Evaluation & Comparison**
   Evaluate all three models on the test set. Report: accuracy, precision, recall, F1-score (macro & weighted), confusion matrix, and inference time per batch. Produce a unified comparison table.

8. **[S] Robustness Analysis**
   Test model robustness under at least two distribution shifts (e.g., Gaussian noise, brightness/-contrast changes, JPEG compression). Plot performance degradation curves. Discuss findings.

9. **[?] Error Analysis**
   Identify the top-k misclassified samples for each model. Visualize with predicted vs. true labels. Discuss common failure patterns and potential remedies.

3.6. Notebook 2 — Transformer Models

[NB] Notebook 2: Transformer-Based Architectures — ViT · Swin · DeiT

Required Section Structure (in order):

1. **[*] Global Configuration** (`START HERE`)
   Define all hyperparameters, patch sizes, positional embedding types, paths, and seeds in a single configuration cell.

2. **[+] Setup & Imports**
   Install `timm` or equivalent transformer library. Import all dependencies and verify GPU. Print `timm` version and available model variants.

3. **[=] Exploratory Data Analysis (EDA)**
   Same requirements as Notebook 1. Additionally, comment on how dataset characteristics (image resolution, class structure) may influence patch-based tokenization.

4. **[W] Data Preprocessing & Augmentation**
   Transformer models typically require higher resolution inputs. Define appropriate transforms. Include at least: resize to 224×224 (or model-specific size), normalization with ImageNet stats, and advanced augmentation suitable for vision transformers (e.g., RandAugment, label smoothing).

5. **[T] Training Infrastructure**
   Implement a training loop with: AdamW optimizer with weight decay, warmup + cosine LR schedule (recommended for transformers), gradient clipping, mixed-precision training (`torch.cuda.amp`), and checkpoint management.

6. **[TF] Transformer Models: ViT · Swin · DeiT**
   For **each** of the three architectures:
   - Load pretrained weights from `timm` and adapt the head.
   - Briefly describe the architectural distinction (e.g., patch embedding, shifted windows, distillation token).
   - Report trainable parameters and `FLOPs` (if possible).
   - Train and plot loss/accuracy curves.
   - Save the best checkpoint.

7. **[ ] Evaluation & Comparison**
   Same metrics as Notebook 1. Additionally, visualize **attention maps** for at least 5 sample images per model to qualitatively interpret what each model attends to.

8. **[S] Robustness Analysis**
   Apply the same distribution shifts as Notebook 1 for fair cross-architecture comparison. Additionally, evaluate sensitivity to input resolution changes.

9. **[?] Error Analysis**
   Identify misclassified samples. For transformer models, overlay attention maps on misclassified images to investigate whether incorrect attention contributes to errors.

4. Cross-Architecture Comparison Report

In addition to the two notebooks, you must include a **brief written comparison** (minimum 300 words) as a markdown cell at the end of *either* notebook (clearly labeled). This comparison should address:

- Overall performance: which architecture family performed better on your dataset, and why?

- Computational trade-offs: training time, number of parameters, inference latency.
- Robustness: which models were more robust to distribution shifts?
- Practical recommendation: for your specific dataset and task, which single model would you deploy, and why?

5. Marking Rubric

Component	Criteria	Marks
Notebook 1 — CNN Models		**35**
EDA	Quality and depth of analysis, visualizations	5
Preprocessing	Justification, completeness of pipeline	5
Model Implementation	Correct architecture setup, transfer learning	10
Evaluation & Robustness	Metrics completeness, robustness experiments	10
Error Analysis	Quality of insights from misclassification study	5
Notebook 2 — Transformer Models		**35**
EDA + Preprocessing	Same criteria + resolution/tokenization discussion	8
Model Implementation	Architecture descriptions, attention map generation	12
Evaluation & Robustness	Metrics, attention-map-based error analysis	10
Training Setup	Correct use of AdamW, warmup, amp	5
Cross-Architecture Comparison		**10**
Comparison Report	Depth, accuracy, practical recommendations	10
Viva (31 March 2026)		**20**
Model Understanding	Ability to explain architectural choices	8
Results Interpretation	Ability to discuss findings & anomalies	7
Q&A	Response to instructor questions on methodology	5
Total		**100**

Table 1: Marking rubric for CSE 475 Programming Assignment

6. Submission & Viva Details

[!] Important Deadlines & Submission Details

Submission (31 March 2026, 11:59 PM)

Step-by-step submission process:

1. **Finalize your notebook** — ensure all cells run top-to-bottom without errors, all outputs are visible, and the group details cell is complete.
2. **Save & Commit** — click **Save & Run All (Commit)** in the Kaggle notebook editor. Wait for the full run to complete successfully. A notebook that has not been committed will not be accepted.
3. **Make the notebook public** — after committing, go to notebook **Settings** and set the visibility to **Public**.
4. **Copy the public Kaggle link** — the URL will follow this format:
   `https://www.kaggle.com/code/<username>/<notebook-title>`
5. **Submit both notebook links** by emailing the instructor with the subject line:
   `[CSE475][Group XX] Assignment 01 - Notebook Links`
   Include: Group ID, both member names & IDs, CNN notebook URL, Transformer notebook URL.

- Notebooks that are **private, uncommitted, or not runnable** at the time of evaluation will receive **zero marks**.
- Late submissions will incur a **10% penalty per day** (max 3 days; 0 marks thereafter).
- Do **not** modify the notebook after the submission deadline.

Viva (31st March 2026 — Schedule – Starting at 1:30 pm)

- Viva slots will be communicated at least 3 days in advance.
- You must be able to **run and explain any cell** in your notebook on demand.
- Inability to explain your own code will result in significant mark deduction.
- Both group members must attend and demonstrate individual understanding.
- **Viva questions will focus on: model architecture, training decisions, results interpretation, and robustness findings.**

7. Academic Integrity

[=] Academic Integrity Policy

All submitted work must be your own. You may refer to public Kaggle kernels, documentation, and tutorials for *learning purposes only*. Copying code directly from external sources without attribution, or submitting the same notebook as another group, constitutes academic misconduct and will result in zero marks for all parties involved, in addition to disciplinary action as per EWU policy.

AI-generated code (e.g., ChatGPT, Copilot) is **permitted as a learning aid**, but you must be fully able to explain every line of your submitted notebook during the viva.

8. Recommended Resources

- PyTorch Documentation — `pytorch.org/docs`
- timm Library — Pretrained vision models (`timm.fast.ai`)
- ViT Paper — Dosovitskiy et al., 2020 (`arXiv:2010.11929`)
- Swin Transformer — Liu et al., 2021 (`arXiv:2103.14030`)
- DeiT Paper — Touvron et al., 2020 (`arXiv:2012.12877`)
- EfficientNet Paper — Tan & Le, 2019 (`arXiv:1905.11946`)
- Kaggle Learn: Computer Vision Course